

**KÜTAHYA SAĞLIK BİLİMLERİ ÜNİVERSİTESİ
MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ**

9 Nisan 2025



BAP Sistemi Hakem Atama Modülü

Bünyamin Erdoğan

2218121046

1 Giriş

Bu proje, BAP (Bilimsel Araştırma Projeleri) sistemi için hakem atama sürecini otomatikleştirmek amacıyla geliştirilmiştir. Projenin temel amacı, projeler ile hakemlerin uzmanlık alanları arasındaki benzerlikleri hesaplayarak en uygun hakem eşleştirmesini yapmaktır. Veri seti, TÜBİTAK proje arşivi ve KSBÜ BAPSİS proje arşivinden manuel olarak toplanmıştır. Hakem veri seti ise tamamen sentetik olarak oluşturulmuş ve sistemde kullanılmıştır. Bu durum, model çıktılarının analizinde önemli bir faktör olarak ele alınmıştır.

2 Veri Toplama ve Ön İşleme

2.1 Veri Toplama

Projemizde kullanılan hakem verileri, gerçek bir veri setinden veya hazır bir kaynaktan alınmamıştır. Tüm veriler, projenin ihtiyaçlarını karşılayacak şekilde manuel olarak oluşturulmuştur. Ayrıca kullandığımız proje içerikleri verileri ise Tubitak Proje Arşivi[?]e KSBÜ BAPSİS proje arşivi internet sitelerinden el ile kopyalanmıştır. Bu yaklaşımı seçmemizin temel nedeni, sistemin test edilmesi ve algoritmanın doğrulanması için uygun bir test verisi seti elde etmektir.[2][7]

Oluşturduğumuz veri setleri iki ana kategoriden oluşmaktadır:

2.1.1 Projeler Veri Seti

Projeler veri setimizde şu alanlar bulunmaktadır:

- Proje ID: Benzersiz tanımlayıcılar
- Başlık: Çeşitli akademik alanlardan proje başlıkları
- Anahtar Kelimeler: Projeleri tanımlayan anahtar kelimeler

2.1.2 Hakemler Veri Seti

Hakemler veri setimizde şu alanlar bulunmaktadır:

- Hakem ID: Benzersiz tanımlayıcılar
- İsim: Akademik unvanlar ve isimler
- Uzmanlık Alanları: Farklı akademik disiplinler
- Geçmiş Proje Başlıkları: Değerlendirilmiş proje örnekleri

2.2 Veri Ön İşleme

Oluşturulan veriler üzerinde, word embedding kısmına temel olması amacıyla kapsamlı bir ön işleme süreci uygulanmıştır. Bu süreçte Python dilinde sıkça kullanılan nltk (Natural Language Toolkit) kütüphanesinden faydalanılmıştır.[4]:

- Tüm metinler küçük harfe dönüştürüldü
- Noktalama işaretleri ve sayısal değerler temizlendi
- Gereksiz boşluklar düzenlendi
- Özel karakterler standardize edildi

İşlenmiş veriler, CSV formatında saklanmaktadır. Orijinal veriler ise korunmaktadır. Tüm dosyalar UTF-8 karakter kodlaması kullanılarak, Türkçe karakter desteği korunarak kaydedilmiştir.

3 Word Embeddings

Veri ön işleme adımının ardından, metin verilerini benzerlik hesaplamasında kullanabilmek adına kelimelerin sayısal temsillerine ihtiyaç duyulmuştur. Bu amaçla, kelimeler arasındaki anlamsal ilişkileri koruyarak her birini çok boyutlu vektörlerle temsil eden word embeddings yöntemlerine başvurulmuştur. Bu çalışmada iki farklı gömme yaklaşımı kullanılmıştır: biri derin öğrenme tabanlı MiniLM modeli, diğeri ise istatistiksel FastText modelidir.

3.1 MiniLM Modeli

MiniLM, BERT (Bidirectional Encoder Representations from Transformers) mimarisine dayalı, daha hafif ve hızlı bir varyanttır. Bu çalışmada kullanılan MiniLM modeli, Sentence-BERT (S-BERT) yapısı üzerine kuruludur. S-BERT, cümle düzeyinde anlam karşılaştırmaları yapabilen, transformer tabanlı bir dil modelidir. Özellikle metin benzerliği, kümeleme, bilgi çıkarımı gibi görevlerde başarılı sonuçlar vermektedir.

Bu model, metinleri anlamlı vektörlere dönüştürürken her bir kelimenin bağlam içindeki yerini dikkate alarak daha derin anlam yakalamayı mümkün kılmıştır. Model, Hugging Face üzerinden önceden eğitilmiş halde alınmış ve sentence-transformers kütüphanesi aracılığıyla kullanılmıştır.[6]

MiniLM modelinin başarısının arkasında yatan yapı Self-Attention mekanizmasıdır. Self-Attention mekanizması, her kelimenin cümledeki diğer kelimelerle olan ilişkisini anlamaya çalışır. Bunun için her kelimeyi Query, Key ve Value diye üç farklı vektöre ayırır. **Örnek cümle:** "Kedi ağaca tırmandı."

- **Query (Sorgulayan Kelime):** Her kelime, cümledeki diğer kelimelerle ilişkisini sorgular. Mesela, "Kedi" kelimesi, cümledeki diğer kelimelere bakarak, "ağaç" ve "tırmandı" kelimeleriyle ne kadar ilişkili olduğunu anlamaya çalışır.
- **Key (Anahtar Kelimeler):** Diğer kelimeler, Query kelimelerine cevap vermek için bir anahtar işlevi görür. Örneğin, "Ağaç" kelimesi, "Kedi" kelimesine cevap olarak bir anahtar gibi çalışır ve "tırmandı" eylemiyle nasıl ilişkili olduğunu gösterir.
- **Value (Değer):** Son olarak, her kelime kendine ait bir Value değeri taşır. Bu değer, kelimenin anlamını temsil eder. "Kedi" kelimesinin değeri, bir hayvanı, "ağaç" kelimesinin değeri ise bir nesneyi temsil eder.

Self-Attention, bu üç vektörü kullanarak, kelimeler arasındaki bağları anlamaya çalışır. Yani, "Kedi" kelimesi "Ağaç" kelimesiyle nasıl ilişkili olduğunu ve "Tırmandı" kelimesiyle nasıl bağlantılı olduğunu değerlendirir. Böylece cümledeki her kelime, anlamını daha doğru bir şekilde öğrenir.

3.2 FastText Modeli

FastText, Facebook AI Research tarafından geliştirilen, kelime düzeyinde *word embeddings* oluşturan bir modeldir. *Word2Vec* gibi klasik modellerin ötesine geçerek kelimeleri sadece tek bir bütün olarak değil, alt-kelimelere (*subword units*) ayırarak öğrenir. Bu özellik, özellikle Türkçe gibi eklemeli dillerde büyük avantaj sağlamaktadır.

- **Alt-kelimeler üzerinden öğrenme:** Örneğin "doktorlar" kelimesi, "dok", "tor", "lar" gibi parçalara ayrılır ve bu parçaların vektörleri üzerinden genel kelime vektörü hesaplanır.
- **Düşük frekanslı kelimelere karşı dayanıklı:** Nadir geçen kelimeler bile alt parçalardan öğrenilebildiği için anlamlı temsiller elde edilir.
- **Kelime düzeyinde gömme:** Her bir kelime, sabit boyutta ve anlamsal açıdan zengin bir vektör ile temsil edilir.

Bu projede Türkçe için önceden eğitilmiş FastText modeli kullanılmıştır (`cc.tr.300.bin`).[5] Her kelime bu model aracılığıyla 300 boyutlu bir vektöre dönüştürülmüş ve metin düzeyinde ortalama alma yöntemiyle doküman temsilleri oluşturulmuştur.

FastTextin başarısının altında yatan model ise CBOW modelidir. FastText'in kullandığı bir başka önemli model olan CBOW (Continuous Bag of Words), kelimeleri bağlamlarıyla tahmin etmeye çalışır. Bu modelde, çevredeki kelimeler (bağlam kelimeleri) alınarak, hedef kelime tahmin edilmeye çalışılır. Örneğin, "Kedi ağaca tırmandı" cümlesinde, "Kedi" ve "Ağaca" kelimelerinin vektörleri birleştirilir ve bu bağlamla "Tırmandı"

kelimesi tahmin edilmeye çalışılır. Model, bağlam kelimelerinin ortalama vektörlerini kullanarak hedef kelimenin vektörünü doğru şekilde tahmin etmeye çalışır ve bu süreçte modelin hatalarını minimize etmek için geriye yayılım (backpropagation) yöntemiyle ağırlıklar güncellenir[1]. Bu sayede, bağlamdaki kelimelerle ilişkili doğru vektörler öğrenilir.[8]

4 Cosine Similarity

Kelime gömme (embedding) adımlarının ardından, belgeler veya metin parçaları arasındaki anlamsal benzerliği ölçmek için Cosine Similarity (Kosinüs Benzerliği) yöntemi kullanılmıştır. Bu yöntem, iki vektör arasındaki açıyı baz alarak benzerlik hesaplayan, özellikle metin madenciliği ve doğal dil işleme (NLP) uygulamalarında sıkça tercih edilen bir tekniktir.

4.1 Cosine Similarity Nedir?

Cosine Similarity, iki vektör arasındaki açının kosinüsünü hesaplar. Bu değer -1 ile 1 arasında bir skordur:

- 1 değeri, iki vektörün aynı yönü gösterdiğini (yani tamamen benzer olduğunu),
- 0 değeri, iki vektörün birbirine dik olduğunu (ilişkisiz),
- -1 değeri ise zıt yönleri gösterdiğini (tamamen zıt) ifade eder.

Ancak doğal dil işleme uygulamalarında elde edilen embedding vektörleri genellikle negatif olmayan sayılardan oluştuğu için, pratikte *Cosine Similarity* skoru genellikle 0 ile 1 arasında değişmektedir.

Matematiksel Formülü;

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

Burada:

- $A \cdot B$: A ve B vektörlerinin skaler çarpımı,
- $\|A\|$: A vektörünün normu,
- $\|B\|$: B vektörünün normu,
- $\cos(\theta)$: A ve B arasındaki açıya karşılık gelen kosinüs değeri.

Cosine Similarity hesaplaması için *scikit-learn* kütüphanesinin `sklearn.metrics.pairwise.cosine` fonksiyonu kullanılmıştır. Bu fonksiyon, verilen iki vektör arasındaki benzerliği hesaplamak için etkili bir yöntem sunar. `cosine_similarity` fonksiyonu, veri noktalarındaki tüm

çiftler için benzerlik skorları üretir ve bu skorlar, iki vektör arasındaki açıyı hesaplamak için kullanılan formülle bağlantılıdır. Bu yöntem, çok büyük veri setlerinde dahi hızlı ve verimli bir şekilde çalışabilmektedir.[3]

5 Karşılaşılan Hatalar

Bu bölümde, proje sürecinde karşılaşılan teknik zorluklar ve bu sorunları çözmek için uygulanan yöntemler açıklanmaktadır. Çalışma boyunca hem kullanılan modeller hem de geliştirme ortamıyla ilgili çeşitli sorunlarla karşılaşmıştır.

5.1 No module named "fasttext" Hatası

Projede FastText modelini kullanmaya çalışırken, "pip install fasttext" komutunu kullanmış olmama rağmen, No module named 'fasttext' hatasıyla karşılaşıldı. FastText, genellikle Linux ve macOS için tasarlanmış bir kütüphane olduğundan, Windows işletim sisteminde kurulumu ve çalıştırılması zorlu olmuştur. Sorunun çözümü için aşağıdaki adımlar izlenmiştir:

- **Wheel Dosyasının Elle Kurulumu:** Doğrudan pip ile kurulum mümkün olmadığından, https://github.com/mdrehan4all/fasttext_wheels_for_windows adresinden uyumlu bir wheel dosyası indirilerek manuel kurulum yapılmıştır.
- **Python Sürüm Uyumu:** FastText, bazı Python sürümleriyle uyumsuz çalıştığı için Python sürümü 3.13'den 3.11.0'a düşürülmüştür.

Bu adımlarla birlikte, FastText kütüphanesi başarıyla çalıştırılmıştır.

5.2 NameError: name 'init_empty_weights' is not defined Hatası

MiniLM modeli çalıştırılırken NameError: name 'init_empty_weights' is not defined hatasıyla karşılaşmıştır. Bu sorun, kullanılan Hugging Face Transformers kütüphanesinin bazı sürümlerinin belirli model fonksiyonlarını desteklememesinden kaynaklanmıştır:

- **Hata Nedeni:** Modelin çalıştırılmaya çalışıldığı sürümde, gerekli fonksiyon bulunmamaktadır.
- **Çözüm:** Kütüphanelerin uyumlu sürümleri belirlenmiş ve ilgili kütüphaneler, sürüm uyumunu sağlayacak şekilde manuel olarak güncellenmiştir.

6 Gözlemler ve Yorum

Yapılan deneyler sonucunda, FastText modelinin daha tutarlı ve mantıklı benzerlik skorları ürettiği gözlemlenmiştir. Bir çok proje için yapılan hakem atamalarında 2 modelde benzer sonuçlar göstermesine rağmen, tüm atama işlemleri göz önünde bulundurulduğunda FastText modelinin MiniLM modeline kıyasla ufak da olsa daha iyi sonuçlar verdiği çıkarımı yapılmıştır. Bu nedenle, bu projenin genel benzerlik analizlerinde FastText modelinden alınan embedding vektörleri tercih edilmiştir. Sonraki süreçler için alternatif modeller de test edilecektir.

Kaynaklar

- [1] Wikipedia contributors. Backpropagation, 2025.
- [2] Kütahya Sağlık Bilimleri Üniversitesi. Bilimsel araştırma projeleri (bap) koordinasyon birimi. Kütahya Sağlık Bilimleri Üniversitesi Resmi Web Sitesi, 2023. Erişim Tarihi: 9 Nisan 2025.
- [3] Scikit learn Developers. sklearn.metrics.pairwise.cosine_similarity. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html, 2025.
- [4] PythonSpot. Natural language toolkit (nltk) tutorials, 2025.
- [5] Facebook AI Research. fasttext: Pre-trained word vectors for 157 languages. <https://fasttext.cc/docs/en/crawl-vectors.html>, 2018.
- [6] HuggingFace Team. all-minilm-l6-v2 model, 2022. Erişim Tarihi: 2025-03-26.
- [7] TÜBİTAK. Proje arşivi. TÜBİTAK Resmi Web Sitesi, 2023. Erişim Tarihi: 2025-03-26.
- [8] Analytics Vidhya. Continuous bag of words (cbow), 2024.